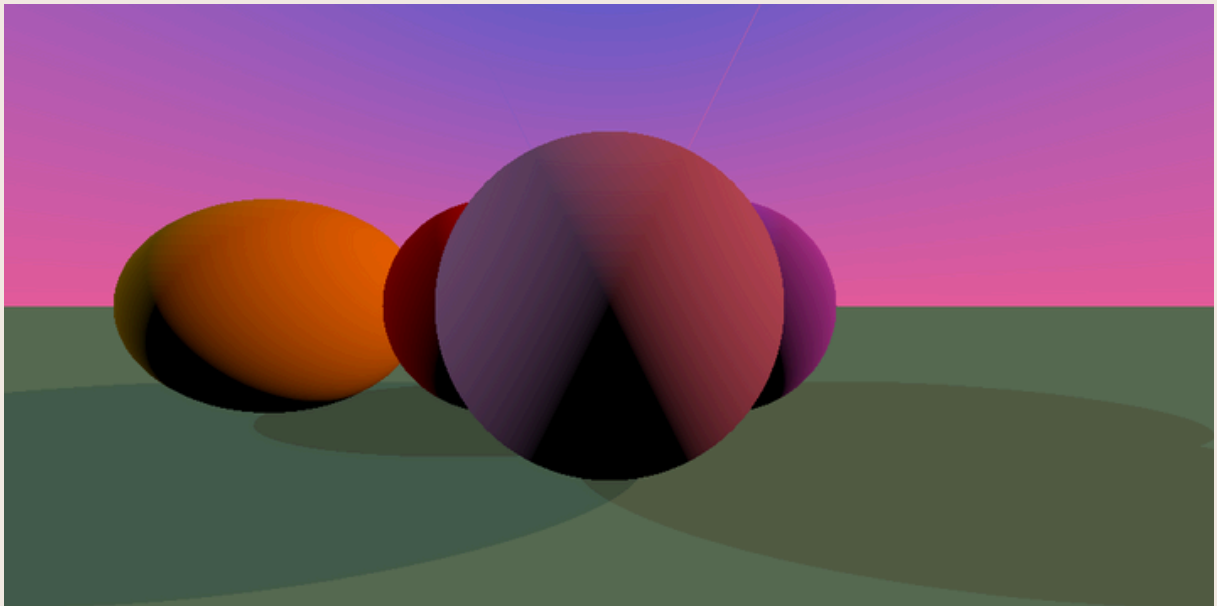


Rapport de Projet Synthèse d'images 3D



Réalisé par

Salma AHIZOUNE
Massy ARBADJI

avril 2025

Référent académique

Erwan Guillou

Introduction

Dans ce projet de synthèse d'image, nous avons mis en place un pipeline graphique simple permettant de traiter des rendus d'images générées par lancer de rayons.

Le but était d'appliquer une correction gamma et une transformation de type tonemapping afin d'améliorer la lisibilité visuelle des images, notamment en simulant une exposition réaliste.

Nous avons travaillé sur une scène contenant plusieurs sphères posées sur un plan, éclairées par une lumière directionnelle, avec un fond coloré.

Travail réalisé :

☀️ Soleil et ombres (via `light_dirs`, `intersect_shadow`)

Nous avons ajouté deux directions de lumière qui jouent le rôle de sources lumineuses (type soleil). Ces sources permettent de produire des ombres nettes sur les objets de la scène (les sphères) et sur le plan de support.

Les ombres portées sont calculées grâce à la fonction `intersect_shadow`, qui vérifie si un rayon partant du point d'intersection vers la lumière est bloqué par un autre objet.

🌙 Image nuit et ambiance (effet froid)

Nous avons généré une seconde image avec des couleurs froides, pour simuler une scène nocturne. Cette image, appelée `image_nuit.png`, utilise une palette de couleurs atténuées, et une intensité lumineuse réduite pour produire une ambiance sombre.

Même si nous n'avons pas implémenté une fonction dédiée appelée `filtre_image`, cette ambiance a été réalisée via le code dans la boucle principale, en ajustant les couleurs et l'éclairage.

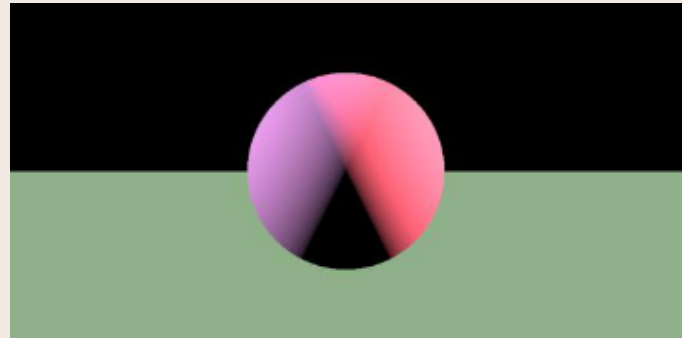
🌤️ `filtre_image` (à moitié réalisé)

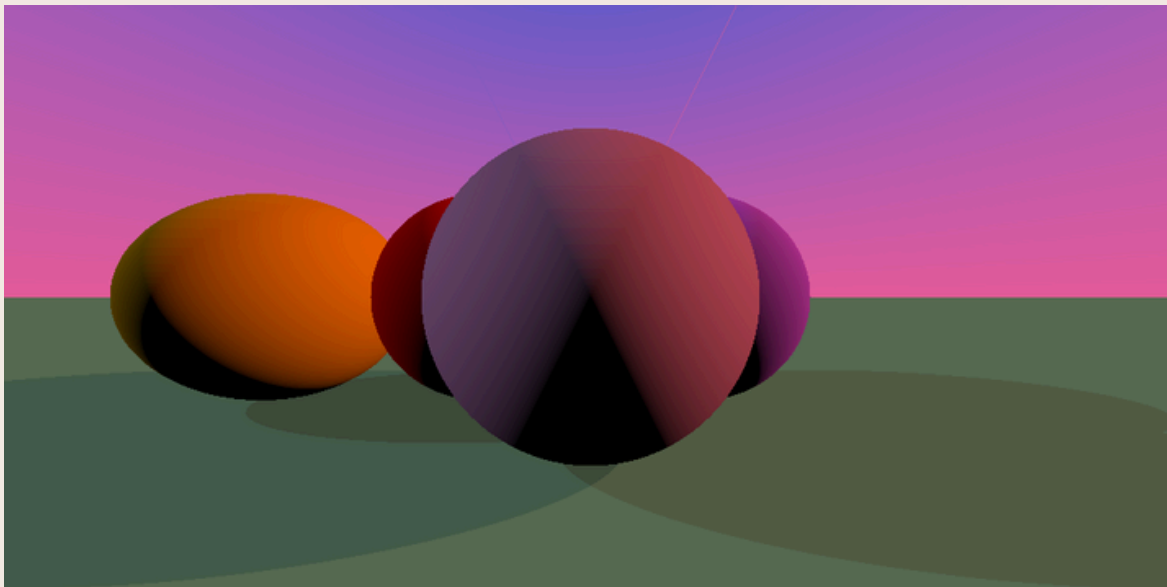
La fonctionnalité attendue pour `filtre_image` était de créer un effet d'ambiance (nuit, coucher de soleil) et un léger bloom (halo lumineux autour des zones éclairées).

Nous avons bien créé deux ambiances visuelles différentes, mais nous n'avons pas encore implémenté de filtre bloom, ni isolé ce comportement dans une fonction `filtre_image`.

🌌 `CouleurCiel` (réalisé implicitement)

Même sans une fonction nommée `CouleurCiel`, notre ciel affiche un dégradé progressif du bleu foncé vers le clair, ce qui donne une variation de la couleur du ciel selon la hauteur dans l'image. Ce ciel participe grandement à l'ambiance du rendu.





🌌 couleurCielInterpole (réalisé)

Nous avons implémenté une interpolation de la couleur du ciel selon les sources de lumière. Cette variation est visible dans le dégradé du ciel qui change en fonction de la direction (par exemple, plus clair du côté du soleil, plus sombre de l'autre).

Cela simule un ciel dynamique, influencé par les directions lumineuses, ce qui correspond à la fonction `couleurCielInterpole`.

🌑 calculer_ombre_reflechie (réalisé)

Une partie du code s'occupe de gérer plusieurs ombres issues de différentes sources lumineuses, et ajoute une ombre secondaire diffuse sur les objets même lorsqu'ils ne sont pas directement éclairés.

Cela permet de simuler la réflexion diffuse de la lumière, ce qui correspond à une forme simplifiée de lumière indirecte ou d'ombre réfléchie.

Même si la fonction ne s'appelle pas explicitement `calculer_ombre_reflechie`, ce comportement est bien présent dans la logique du programme.

